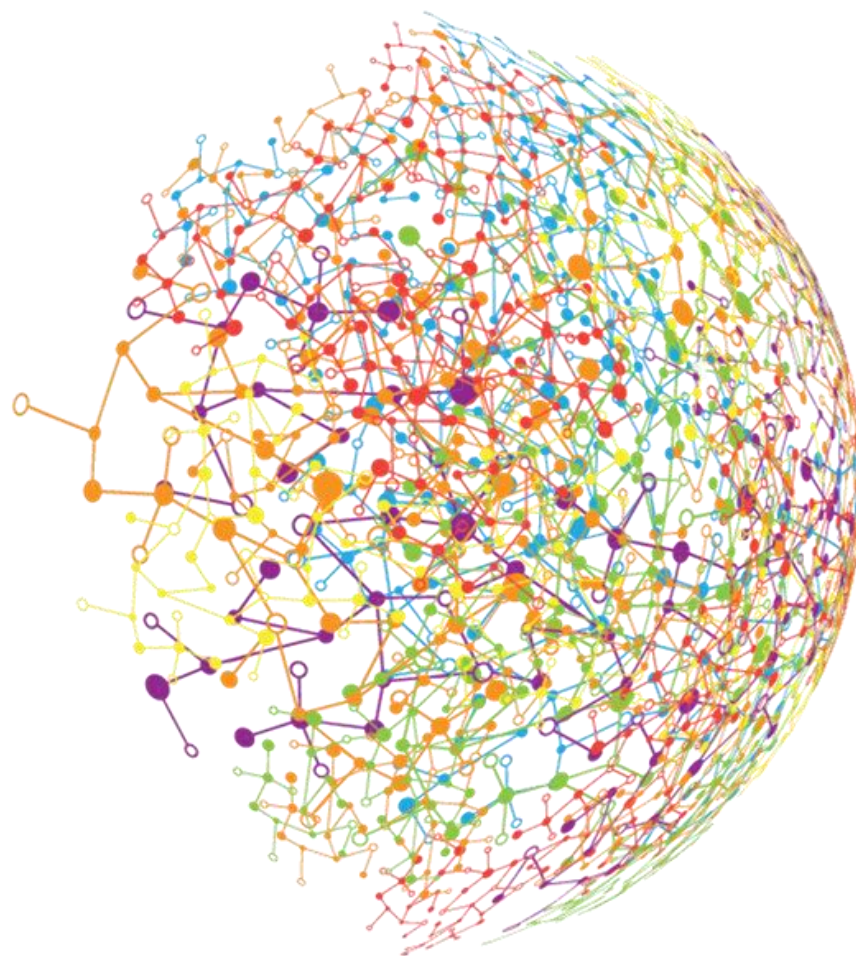




開発・運用環境

2025年11月1日 改訂版

DeepSquare AI講座 AIエンジニア育成講座（E資格講座）

エッジコンピューティング

- エッジコンピューティング



✓ モデルの軽量化

- 分散処理

- アクセラレータ

- 環境構築

モデルの軽量化

目標

- 量子化・プルーニング・蒸留それぞれのモデル軽量化の基礎的な手順と手法ごとのメリットおよびデメリットを把握して、どのような場面で適用すればよいか理解する。

キーワード

- エッジコンピューティング
- IoTデバイス
- プルーニング（枝刈り）
- 蒸留（Distillation）
- 量子化（Quantization）

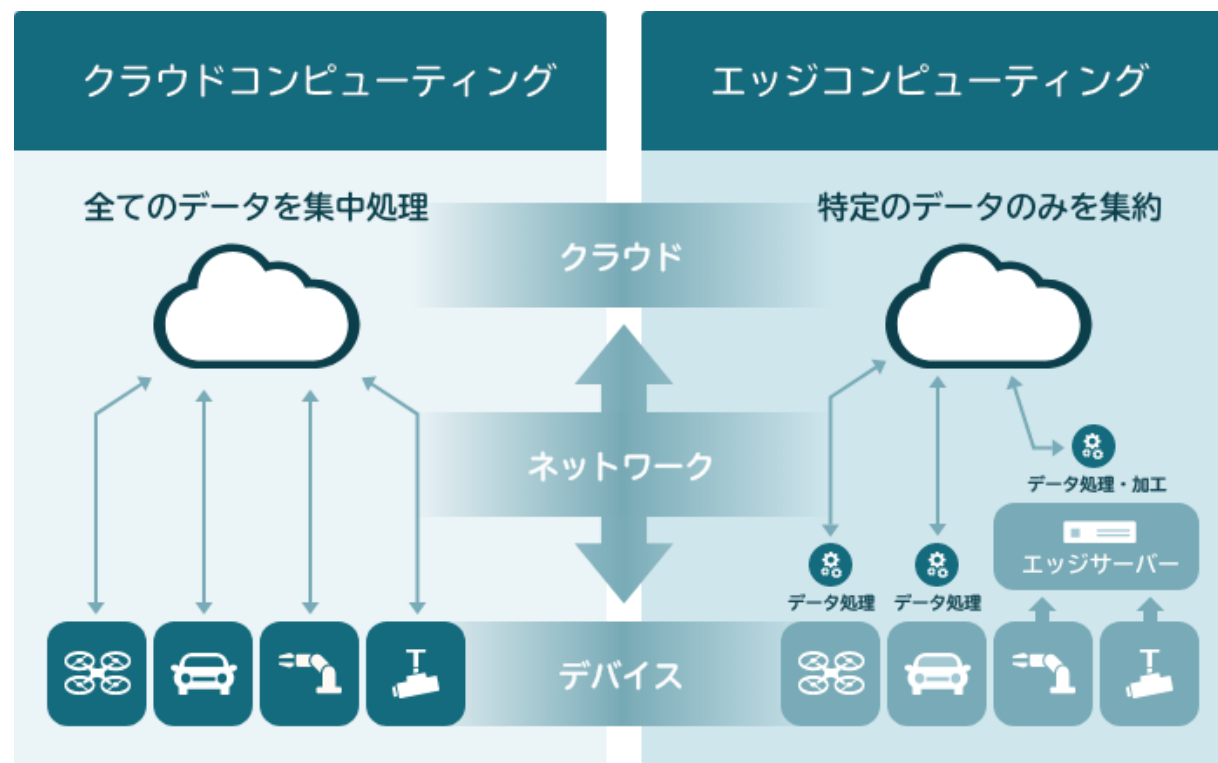
ポイント

- 量子化を行う基本手順と、メリットおよびデメリットを説明できる。
- プルーニングを行う基本手順と、メリットおよびデメリットを説明できる。
- 蒸留を行う基本手順、メリットおよびデメリットを説明できる。

モデルの軽量化

エッジコンピューティング

- 従来のネットワークの処理はサーバーで行うことが主流。
- エッジコンピューティングはデバイスに近い位置でデータ処理を行う。
- リアルタイムでの処理を可能にし、データの送信におけるセキュリティも向上させることができる。



モデルの軽量化

IoTデバイス

- インターネットなどに接続され、データの収集・送信が可能なデバイスのこと。
- 工場や農場での効率的な生産管理を支援することができる。



モデルの軽量化

モデル圧縮の手法

効率的なマイクロアーキテクチャ

- 訓練に先立って、精度と計算のトレードオフを改善するように、モデルを工夫する。
-

量子化

- 小数点数の精度を下げる。
-

Pruning

- 訓練後に、寄与度の小さい重み、チャネル、レイヤを削減する。
-

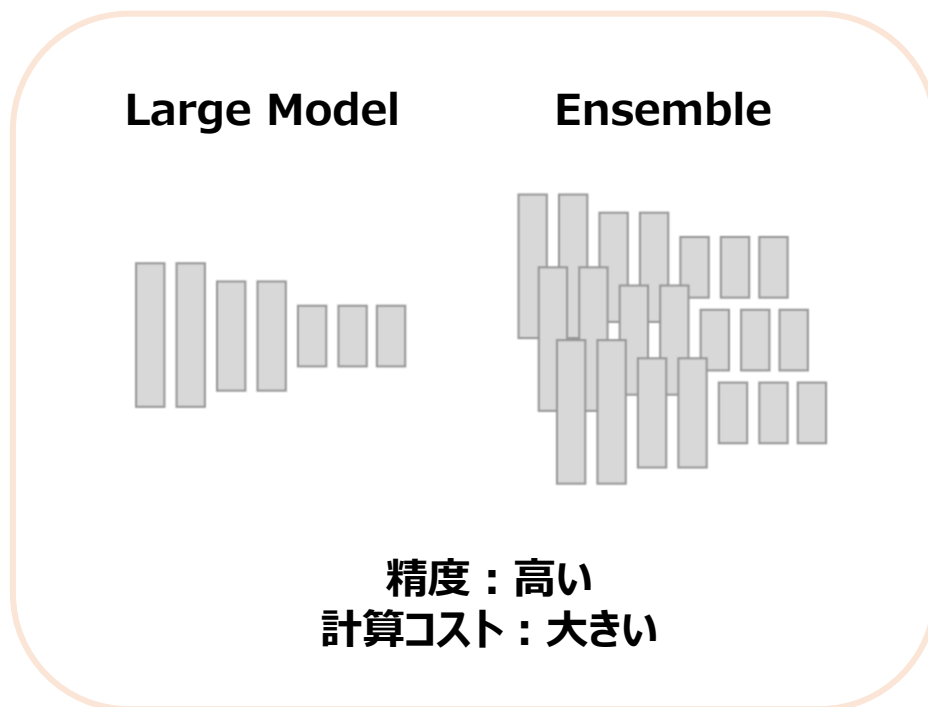
蒸留

- 一度訓練したモデルが学習した知識を、別の軽量なモデルに継承させる。

モデルの軽量化

精度 vs 処理速度

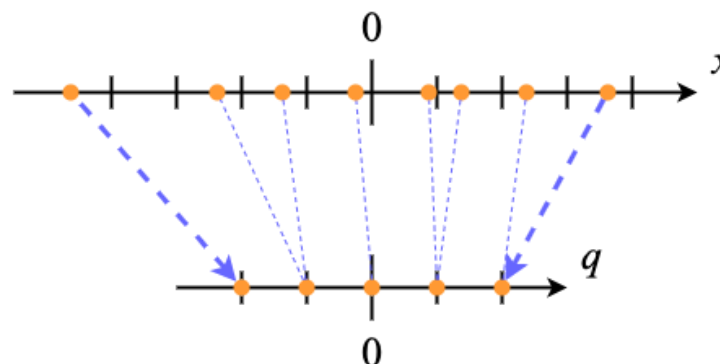
- 一般的に、精度と処理速度はトレードオフになる。（どちらか一方を高めると、もう一方が低くなる。）
- 制限が無ければ、どちらも満たすことが望ましい。（精度が高く、処理速度が速い。）



モデルの軽量化

量子化 (quantize)

- 入力データや重みの小数点数の精度を下げることで、必要なメモリや計算コストを下げる手法。モデルの精度に直結するため、モデルの精度が下がらない範囲で小数点数の精度を下げるような慎重な処理が求められる。



ストレート・スルー・エステメータ (STE: Straight Through Estimator)

- 量子化をすると、0 になる勾配が多く、学習がうまく進まないため、活性化関数を逆伝播する際だけ、恒等写像関数として扱う手法。

モデルの軽量化

Pruning（剪定）

- 学習後に、寄与度の小さい重み、チャネル、レイヤを削減することで省モデル化する。
- なお、Pruningしたままだと精度が下がるため、Pruning後に再学習（Fine-tuning）するのが一般的である。
- ほとんどのプルーニング手法は、既存のモデルアーキテクチャに影響を与えずに適用できる。
- 追加の学習フェーズやパラメータのチューニングが必要になるため、プルーニングによって訓練時間が長くなる場合がある。

正則化項

- ラッソ正則化を用いると重みを0にすることができる。そのため、ラッソ正則化項を用いた剪定も行われる。

宝くじ仮説

- 巨大なモデル内部には、同程度の精度でタスクをこなせるサブネットワークが存在しているという仮説。
- サブネットワークを見つけて実装することで大幅にパラメータを削減することができ、省モデル化につながる。

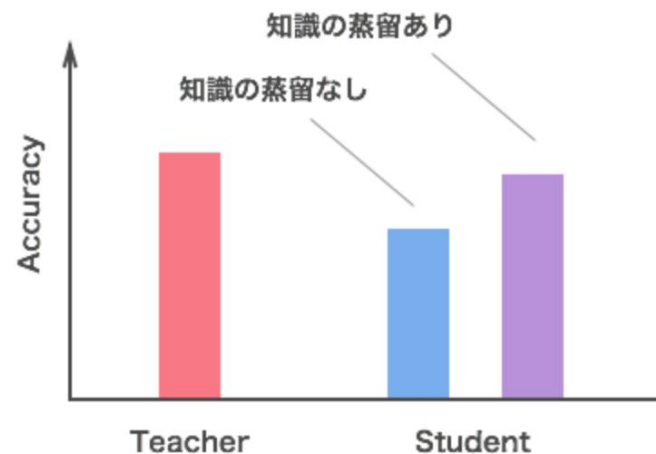
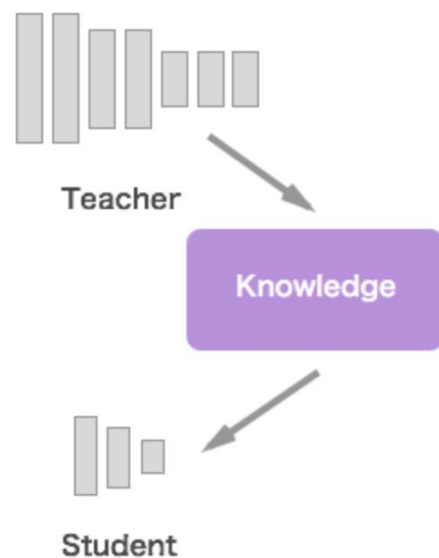
モデルの軽量化

蒸留（distillation）

- 有用な情報だけを利用する（知識の蒸留）ことでモデルを軽量化する手法。

蒸留の手順

- ① 予測精度の良い、大きいモデルやアンサンブルさせたモデルを**教師モデル**として準備する。
 - ② その知識を軽量な **生徒モデル**の学習に利用する。
- ⇒ 軽量でありながら、教師モデルに匹敵する精度のモデルを得ることが期待できる。



※アンサンブルさせたモデル…2つ以上を組み合わせたモデル。

モデルの軽量化

蒸留の発想

- 学習時と推論時に求められるモデルの役割が異なる

学習時

- 大量の学習用データセットから構造化された知識を抽出する。
- 訓練データを丸暗記するのではなく、未知のデータに対する認識精度が良くなるように訓練を行う。

推論時

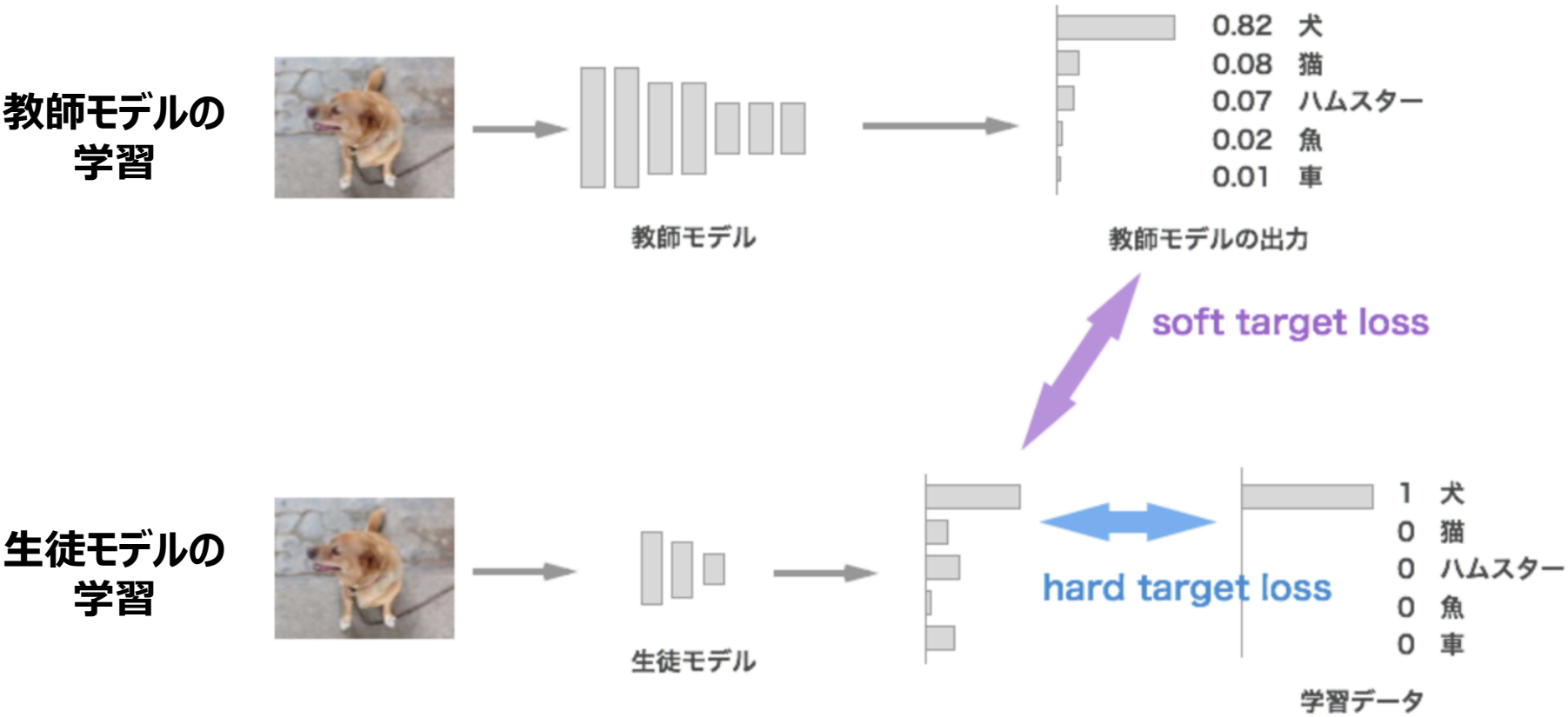
- 限られた計算資源の中で動作する。
- 予測精度だけでなく、処理速度も重要となる。



通常の学習では学習と推論に同じモデルを利用するが、（知識の）蒸留では、**学習データから知識を構造化・一般化する部分は教師モデルが担当**し、その知識を蒸留して**推論に適した生徒モデルに継承させる**、というように役割分担をすることで軽量化を目指している。

モデルの軽量化

蒸留の概要図

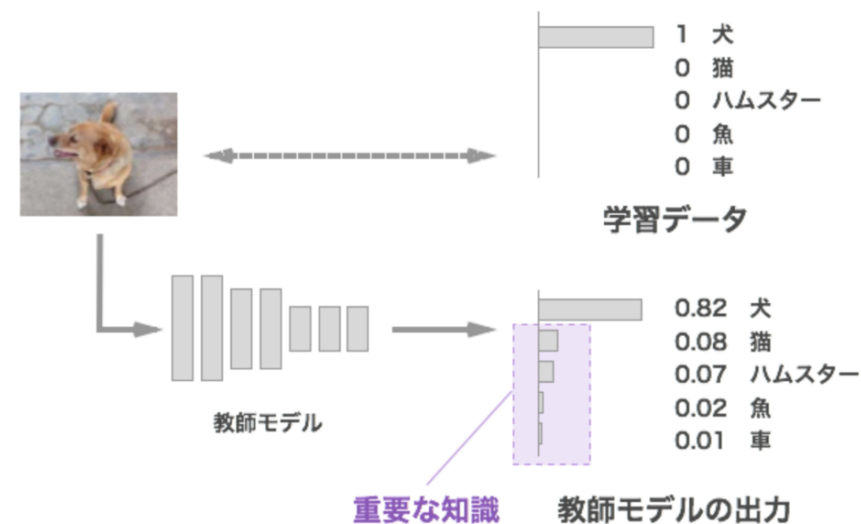


モデルの軽量化

「学習データから得られる情報」と「学習済み教師モデルから得られる情報」の違い

学習データから得られる情報

- 学習データは入力となる画像と正解クラス（図では「犬」）のペアになる。
- 犬が正解ということはわかるが、それ以外のクラスはどれも等しく「正解ではない」という情報しかない。



学習済み教師モデルから得られる情報

- 学習済みの教師モデルは、合計すると 1 になる確率の形で、クラスごとのスコアとして出力される。
- 正解ではないクラスのうち、「猫は少し似てるけど車は全然似ていない」、といったことも、相対的なスコアから知ることができるため、概念的に何と近いのか、何とは全く違うのかといった情報を知ることができる。
⇒学習データよりもより豊かな情報を伝えることができる。

分散処理

- エッジコンピューティング
- 分散処理
- アクセラレータ
- 環境構築



✓ 並列分散処理

並列分散処理

目標

- モデル並列化、データ並列化の考え方および手法を理解する。

キーワード

- 分散深層学習
- モデル並列化
- データ並列化

ポイント

- モデル並列化の概念および特徴を説明できる。
- データ並列化の概念および特徴、同期型と非同期型の手法の違いを説明できる。

並列分散処理

分散深層学習とは

- ディープラーニングを「分散」させて「学習」する手法。
- 分散の方法としては、「複数台のPCをネットワークで繋いで分散させる」、「Multi-GPUを用いる」などの方法がある。

分散深層学習の2つのアプローチ

1. データ並列

- 全プロセスに同じモデルのコピーをして学習することで、バッチサイズをプロセス数倍し、学習を高速化する手法。

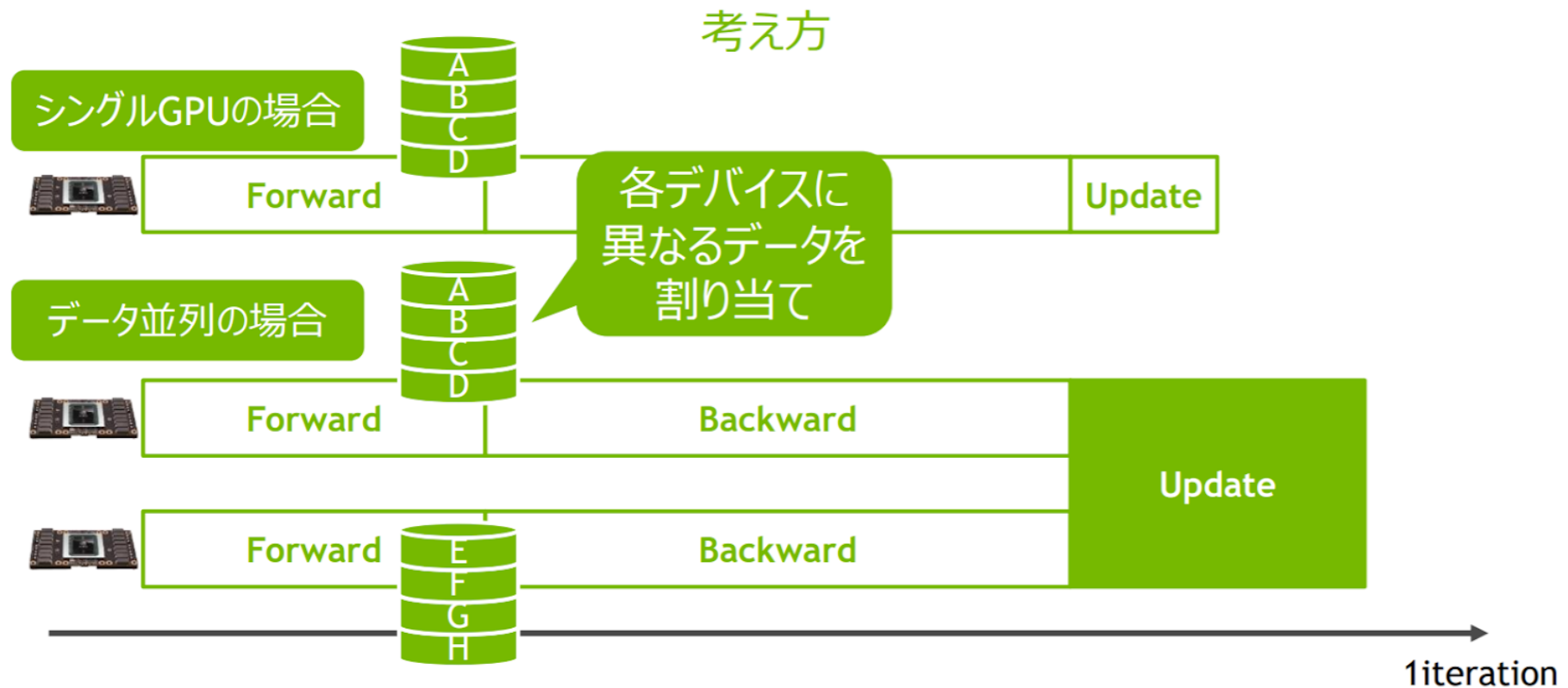
2. モデル並列

- モデルを分散して複数のプロセスに配置し、全プロセスで協調して1つのモデルを学習する手法。
- 1プロセス上に載りきらないサイズのモデルを学習したい場合などに用いられる。

並列分散処理

1. データ並列

- 全プロセスに同じモデルのコピーをして学習することで、バッチサイズをプロセス数倍し、学習を高速化する。



並列分散処理

1. データ並列

- モデルをコピーして、パラメータを共有した状態でそれぞれ別のサンプルで学習を行う。
- GPUごとに別々のデータで勾配を計算させるので、モデル更新時にはパラメータを一致させる必要がある。

同期更新と非同期更新

- パラメータの更新方法は、同期更新と非同期更新に分けられる。

同期更新

- 全GPUの勾配計算完了を待ってモデルを更新する。
- GPU性能に差がある場合、無駄が生じる。

非同期更新

- 一部のGPUの勾配計算が終わったらモデルを更新する。
- Staleness問題（勾配やパラメータ等の情報が古い問題）が発生する。

並列分散処理

データ並列のデメリット

同期更新のデメリット

GPUの性能差で無駄が発生する

- パラメータの更新時にすべてのGPUでの勾配計算が完了するのを待つため、GPUに大きな性能差がある場合に、無駄な時間が発生する。

非同期更新のデメリット

Staleness 問題

- 勾配の計算中にほかのGPUがパラメータ更新するので、勾配計算に使っていたパラメータが相対的に古くなってしまふ（**勾配の陳腐化**）。
- Staleness 問題は収束性の悪化の原因となる。

並列分散処理

分散処理の高速化

ウィークスケーリング

- トータルバッチサイズを増加させて、トータルイテレーション数は減らすことで学習時間を減らすことが目的。
- 1GPUに載せるバッチサイズは変わらない。
- 学習モデルの性能が下がったり、既存研究の実験設定と変わってしまうというデメリットもある。

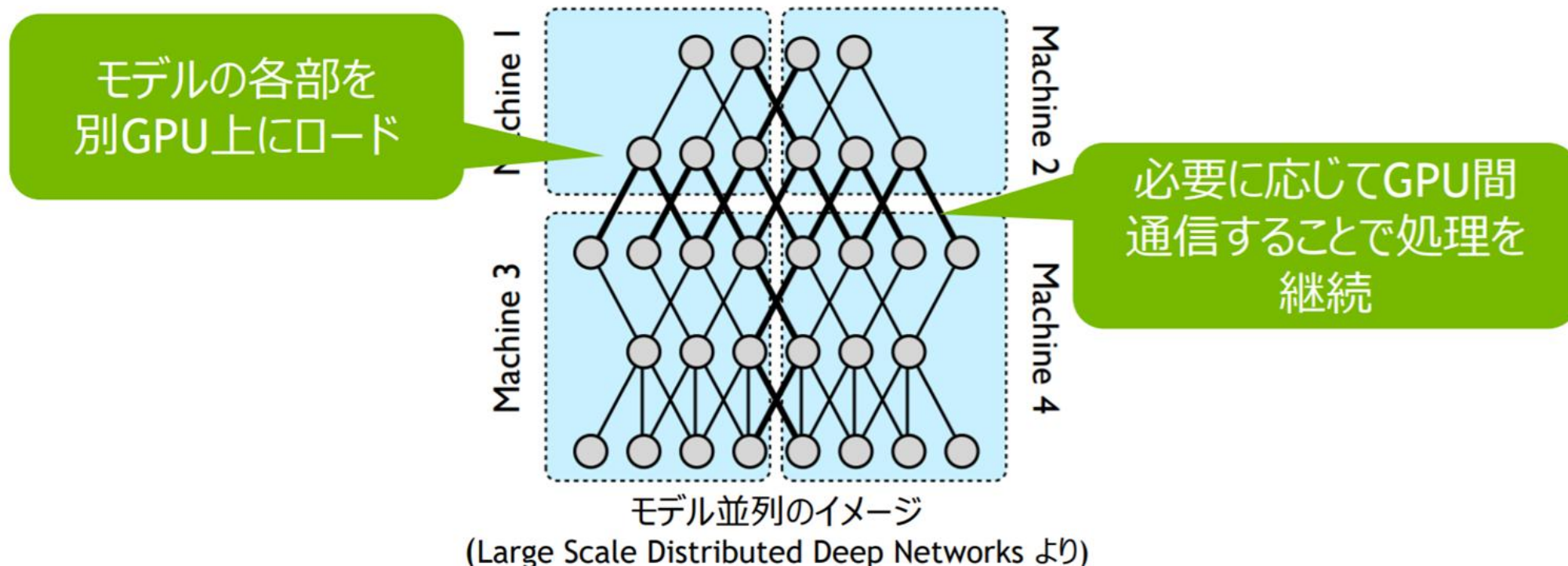
ストロングスケーリング

- トータルバッチサイズは変えずにGPU数で分割したバッチを用いることで1GPUあたりの計算量を減らして学習時間を減らすことが目的。
- 学習モデルの性能や、既存研究の実験設定は変えないが、ある程度1GPUで効率的に計算できるバッチサイズ以下になってしまうとそれ以上高速化はしない。

並列分散処理

2. モデル並列

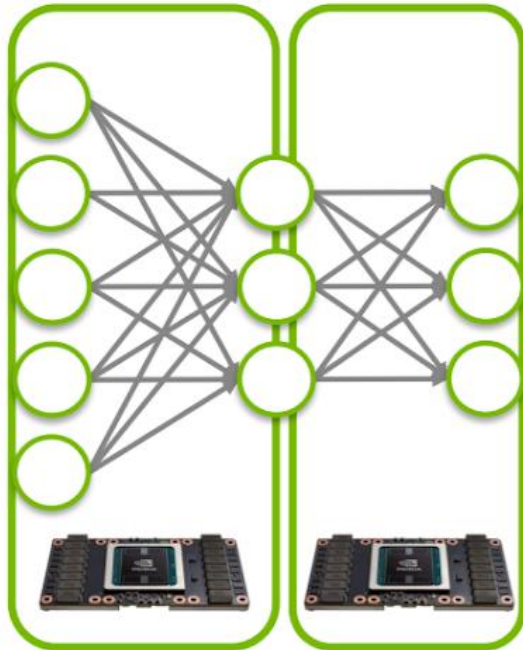
- 複数のサーバーを活用してディープラーニングモデルの学習を行うアプローチ。
- 一台のサーバーでは処理しきれない大きなモデルや複雑なモデルを、複数のサーバーで分散して処理することができる。



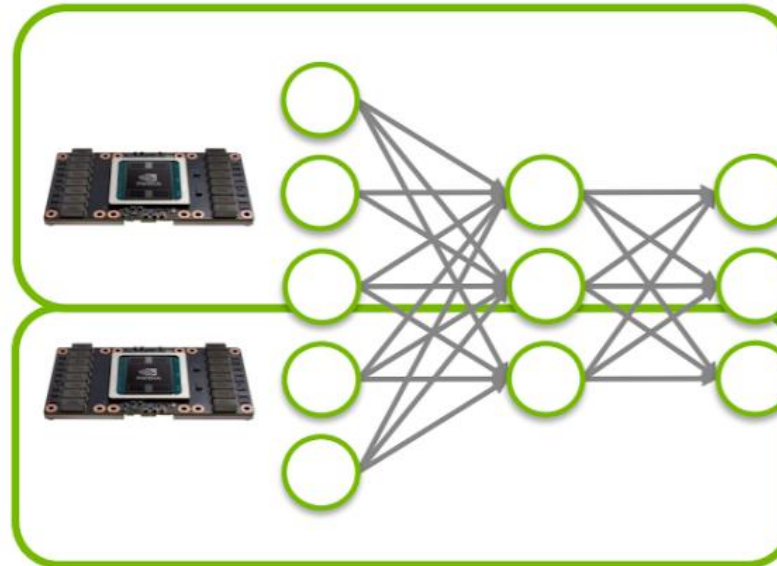
並列分散処理

モデル並列の例

- モデル並列は、「どのようにモデルを分割するか」をタスクやマシンに合わせて検討する必要がある。



各レイヤーを別GPUに割り当て



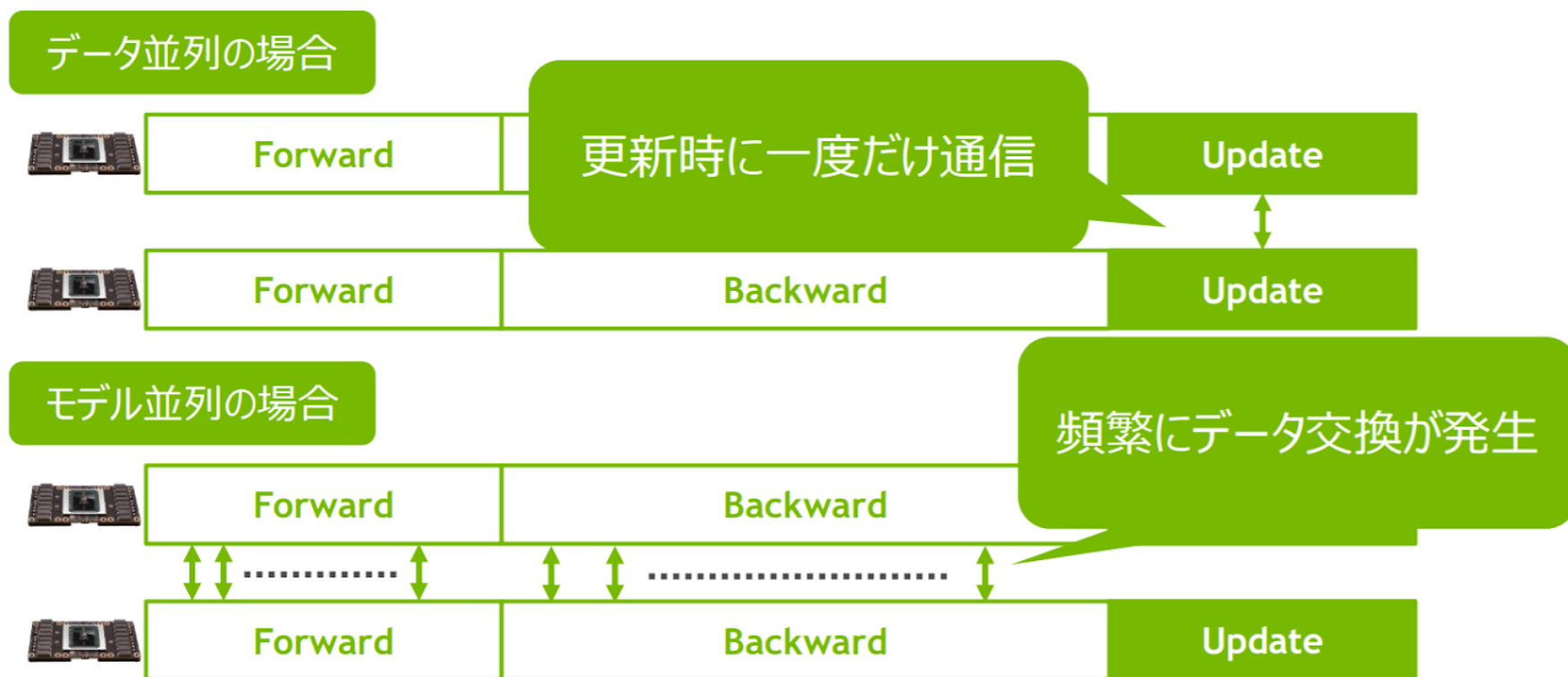
各GPUで各レイヤーの一部を担当

他、畳込みのチャンネルを別GPUに割り当てなどなど.....

並列分散処理

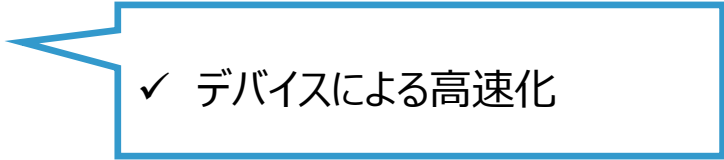
データ並列とモデル並列の比較

- モデル並列は、データ並列と比べて通信コストが増大しやすい。



アクセラレータ

- エッジコンピューティング
- 分散処理
- アクセラレータ
- 環境構築



✓ デバイスによる高速化

デバイスによる高速化

目標

- 深層学習分野における CPU、GPU、TPU の特徴と活用されるケースを理解する。

キーワード

- 単一命令列複数データ (SIMD; Single Instruction Multiple Data)
- 単一命令複数スレッド (SIMT; Single Instruction Multiple Thread)
- 複数命令列複数データ (MIMD; Multiple Instruction Multiple Data)
- GPU (Graphics Processing Unit)
- TPU (Tensor Processing Unit)

ポイント

- CPU、GPU、TPU の特徴と深層学習で利用する際のメリット・デメリットを説明できる。
- 各プロセッサが行列計算を並列に計算する方法を説明できる。
- グラフィック処理が主用途であった GPU が深層学習に用いられるようになった経緯を説明できる。
- GPU、TPU が高速にニューラルネットワークにおける演算を行える仕組みを説明できる。

デバイスによる高速化

並列処理アーキテクチャ

- 並列処理アーキテクチャは、主に 4 種類ある。これらの分類は、フリンの分類（Flynn’s taxonomy）として知られている。

Instruction Pool

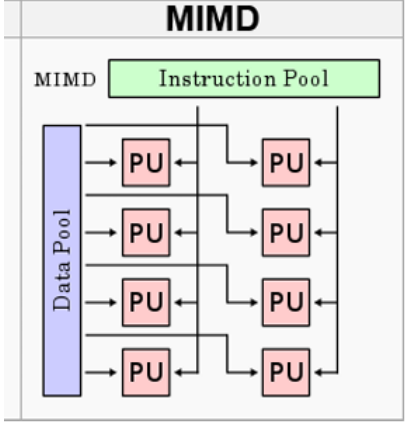
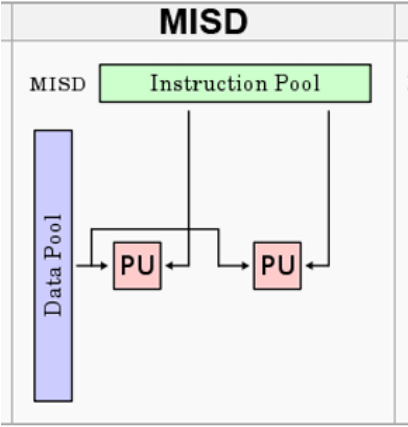
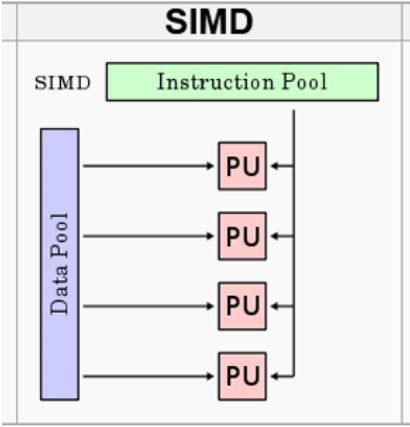
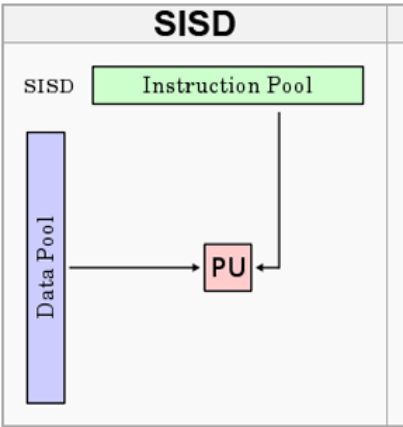
プログラムの命令が格納されている領域。

Data Pool

入力データや処理対象のデータが格納されている領域。

PU (Processing Unit)

実際に計算や処理を実行するユニット。



項目	SISD	SIMD	MISD	MIMD
命令 (Instruction)	単一	単一	複数	複数
データ (Data)	単一	複数	単一	複数
特徴・用途	一般的な逐次処理。 単一の命令が単一のデータを処理。	一つの命令を複数のデータに対して同時処理。並列性が高く、データ並列処理に向く。	理論的なモデルで、実際の使用は特殊ケースのみ。同一データを異なる命令で並列処理。	異なる命令を複数のデータに並列実行。柔軟性が高く、マルチタスク処理に向く。
具体例	通常のCPUによる単純処理	GPUによる画像処理、ベクトル演算	航空宇宙分野などでの冗長性のある検証システム	マルチコアCPUを用いたPCやサーバ

デバイスによる高速化

並列処理アーキテクチャ

SIMT

- フリンの分類の他にも、同じ命令を複数のスレッドが同時に実行する並列処理の実行モデルであるSIMT（Single Instruction, Multiple Threads）がある。
- SIMTは、**1つの命令を複数のスレッドに同時に適用する方式**である。これは、CPUの「SIMD（Single Instruction, Multiple Data）」に似ているが、SIMTの特徴は、各スレッドが独立した制御フローを持つことができるという点にある。
- NVIDIAのGPUなどで採用され、GPUでの行列計算や画像処理、ディープラーニングなど、大量の並列処理が求められる場面で活躍する。

デバイスによる高速化

GPU

- 「**Graphics Processing Unit : 画像演算処理装置**」の略称3D
- グラフィックスなどの画像描写を行う際に必要となる計算処理を行う半導体チップ(プロセッサ)のこと。



GPGPU (General-Purpose computing on Graphics Processing Units)

- 科学技術計算などの映像描画以外の目的で利用されるGPUをGPGPUという。
- 膨大な計算処理を必要とする3Dグラフィックス用途に設計されたプロセッサ・GPUは高い演算性能をもち、その高い演算性能を活用する「GPUサーバー」という考え方が登場した。GPUサーバーというハードウェア上でGPGPUが実行される。
- GPGPUのための統合開発環境としては、NVIDIA社による「**CUDA**」、AMD社の「**AMD Stream (ATI Stream)**」Apple社および業界団体クロノス・グループによる「**OpenCL**」、Microsoft社による「**DirectCompute**」などがある。

デバイスによる高速化

GPUとCPUの違い

共通点

パソコンやサーバーが機能するために必要な「計算処理を行う」という点。

相違点

CPU = 全体を統括する頭脳。複雑な処理、多様な命令、分岐・条件分岐の多いロジックを処理することが得意。
複数コアを使うことでタスクを並行処理できる（タスクのマルチスレッド処理）

GPU = 専門分野を大量に行う頭脳。単純な処理を並列的に大量に行うことが得意。（数百～数千のコアを活用）

	CPU	GPU
主な役割	コンピュータ全体の計算処理	3Dグラフィックスなどの画像描画に必要な計算処理
得意とする計算処理の種類	複雑な処理、多様な命令、分岐・条件分岐の多いロジックを処理すること	大量のコアで同じ種類の演算を一気に並列処理すること
コア数	数個	数千個
計算速度の差	GPUは、CPUの数倍～100倍以上の計算速度を実現することがある。	

デバイスによる高速化

单精度浮動小数点数

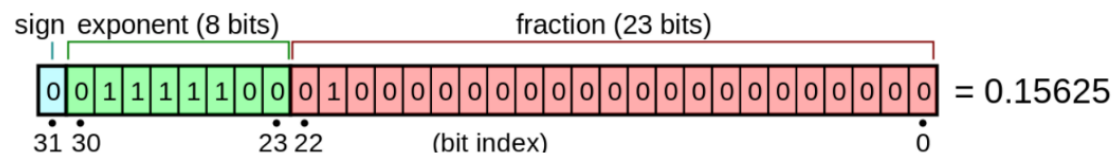
- 32ビットの浮動小数点数表現。

IEEE 754 での binary32 の定義は以下の通りである。

符号ビット: 1ビット

指数部の幅: 8ビット

仮数部の幅: 23ビット



倍精度浮動小数点数

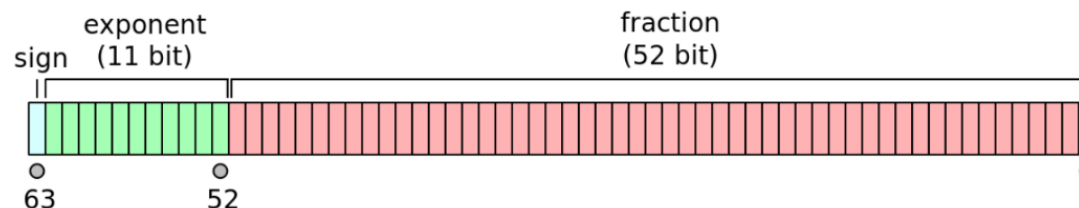
- 64ビットの浮動小数点数表現。

IEEE 754 での binary64 の定義は以下の通りである。

符号ビット: 1ビット

指数部の幅: 11ビット

仮数部の幅: 52ビット



参考：<https://ja.wikipedia.org/wiki/%E5%8D%98%E7%B2%BE%E5%BA%A6%E6%B5%AE%E5%8B%95%E5%B0%8F%E6%95%B0%E7%82%B9%E6%95%B0>
<https://ja.wikipedia.org/wiki/%E5%80%8D%E7%B2%BE%E5%BA%A6%E6%B5%AE%E5%8B%95%E5%B0%8F%E6%95%B0%E7%82%B9%E6%95%B0>

デバイスによる高速化

GPUの単精度と倍精度

	メリット	デメリット
単精度 浮動小数点演算	• 速度: 単精度演算は倍精度よりも速く、高速な計算が可能。 • リソースの効率: 少ないメモリと処理能力で済むため、リソースを節約できる。 • 応用の幅広さ: 特にリアルタイム処理や大規模なデータ処理に適している。	• 精度の制限: 科学計算や高精度が求められるアプリケーションでは不十分な場合がある。
倍精度 浮動小数点演算	• 高精度: より複雑で精度が要求される計算に適している。 • 科学的・技術的計算に最適: 物理シミュレーションや気象予測など、精度が重要な場面で優れている。	• コストと速度: 計算速度が遅く、より多くのリソースを消費する。 • リソースの要求: より多くのメモリと処理能力を必要とするため、コストが高くなる。

- GPUは主に画像処理やゲーム、AIアプリケーションなどの分野で使用されており、これらの分野では計算速度と効率が重要である。単精度演算はこれらの要件に適しており、高速でリソース効率の良い演算を実現する。
⇒GPUにおいては単精度の浮動小数点演算が主流となっている。

デバイスによる高速化

GPUのメモリのエラーの検出と訂正

グラフィック処理の場合

- 1ピクセル程度のずれでは画面表示にほとんど影響が無いため、**グラフィック処理を行うGPUでは、エラー検出や訂正は行われていなかった。**

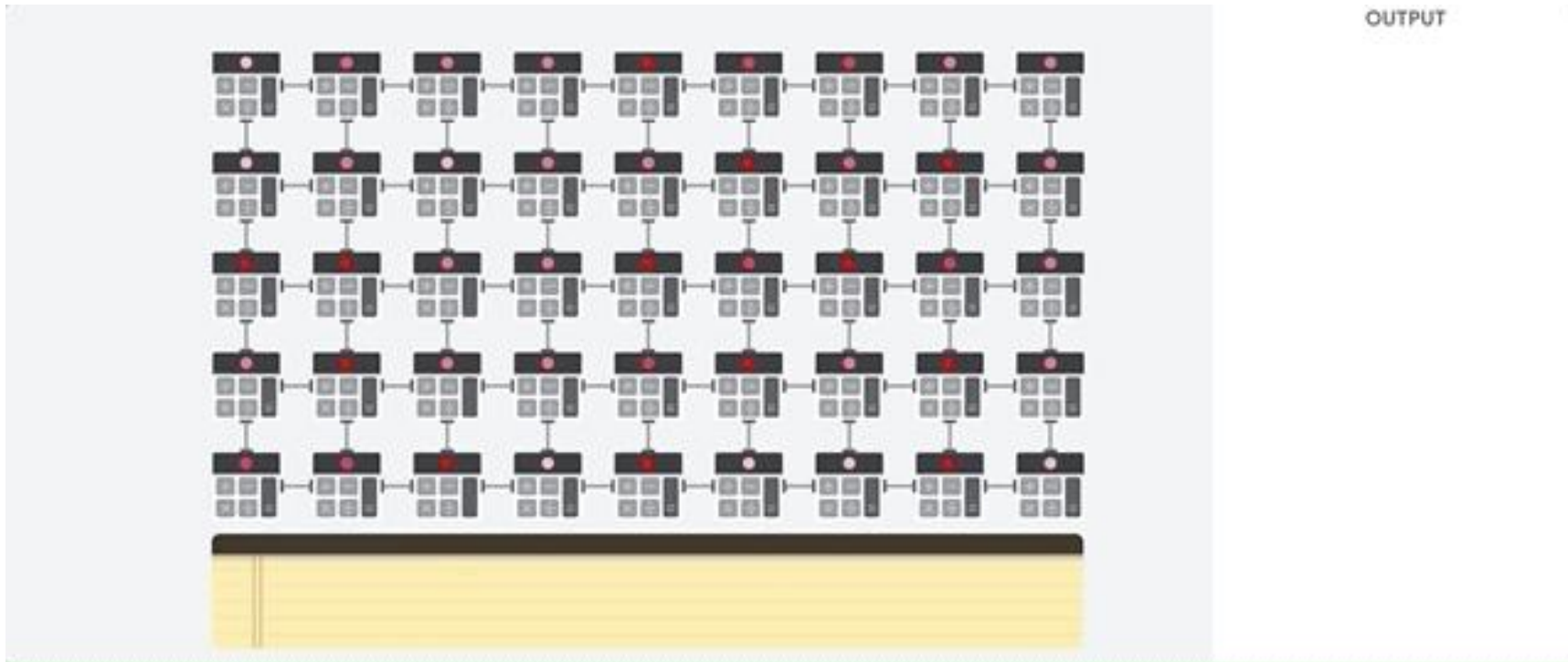
科学計算処理の場合

- GPUを科学計算に使用するようになると、1カ所のエラーが計算結果全体を大きく誤らせてしまうということも起こる。
- NVIDIAのTesla GPUや、AMDのハイエンドのFirePro GPUではGDDR5メモリはエラーの検出、**訂正を行う機能（ECC）**を組み込んでいるものもある。

デバイスによる高速化

TPU (Tensor Processing Unit)

- 特に深層学習のために設計されたGoogleのカスタムASICのこと。
- ニューラルネットワークの計算に頻繁に使用される大規模な行列演算に特化し、多くの演算を並行して処理する能力がある。
- TensorFlowなどのGoogleのフレームワークとの互換性が高く、統合が容易である。



デバイスによる高速化

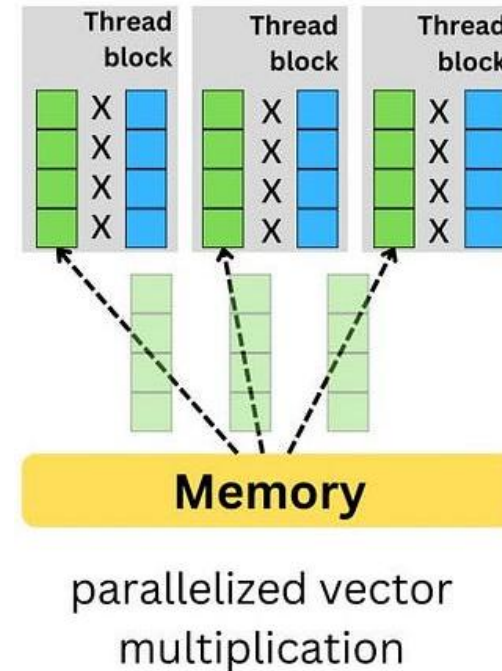
GPUの演算の流れ

- 複数のSIMDユニットが複雑な方法で協調して動作する。
- 画像のピクセルなど多数の小さなベクトルに対して同じ演算を同時に行う。

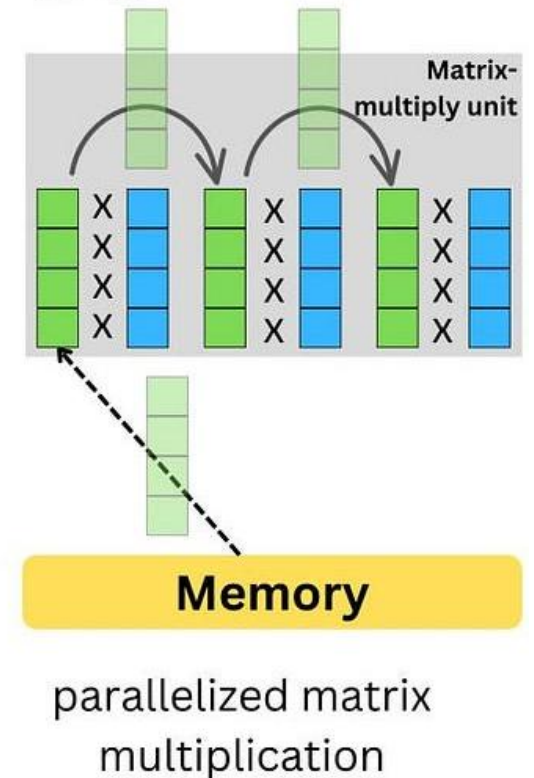
TPUの演算の流れ

- 行列乗算を並列化して実行するように特化している。
- ニューラルネットワークのトレーニングや推論など、大規模な行列演算が必要なタスクにおいて非常に効率的である。

GPU



TPU



コンテナ型仮想化

目標

- 深層学習モデルの開発環境を Docker を用いて構築し、訓練の実施・環境の配布を行えるようになる。

キーワード

- 仮想化環境
- ホスト型
- ハイパーバイザー型
- コンテナ型
- Docker
- Dockerfile

ポイント

- 仮想環境におけるホスト型、ハイパーバイザー型、コンテナ型の特徴を理解する。
- Docker の特徴と基本的な使い方を理解する。
- Dockerfile を記述して深層学習モデルを訓練する環境を構築・訓練・配布できる。

コンテナ型仮想化

仮想化

- 存在していない機器や資源をあたかも存在しているように見せる技術。広義では、ITシステムを構成する物理サーバー、クライアントPC、ストレージなどをソフトウェアによって仮想的に複製し、管理すること。
- 1 台の物理PCや物理サーバー上で複数のOSやアプリケーション実行環境を構築することで、限られた物理的なリソースを効率よく利用することができる。
- 仮想化技術は、大きく「**ハードウェアレベル仮想化（サーバ仮想化）**」と「**OSレベル仮想化**」の2つに分類できる。

ハードウェアレベル仮想化

- 物理的なPCのハードウェアをソフトウェアによって模倣する技術。
- サーバーハードウェアを疑似的に再現した「仮想マシン」を提供する。
- 仮想マシン上に通常のサーバーを利用するのと同じようにゲストOS（仮想マシン上で動いているOS）をインストールして、そのうえでアプリケーションを稼働させる。
- その中でも「**ホスト型仮想化**」と「**ハイパーバイザー型仮想化**」などがある。

OSレベル仮想化

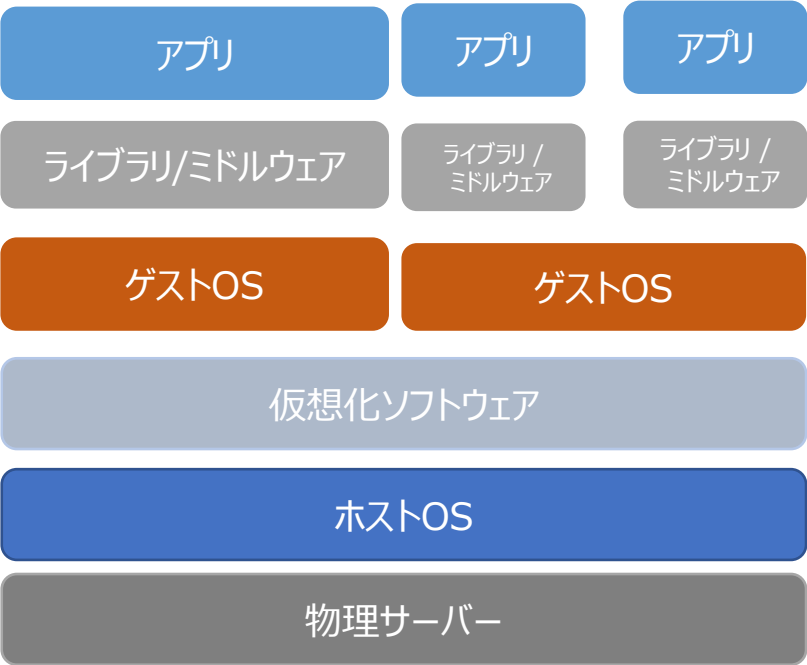
- アプリケーション実行環境を分割した「隔離環境」をホストOS（ベースとなるOS）の機能で提供する。
- 隔離環境内で、外部環境と独立してアプリケーションを実行できる。
- 「**コンテナ型仮想化**」などがある。

コンテナ型仮想化

ハードウェアレベル仮想化

ホスト型仮想化

- ホストOS上で動作する仮想化ソフトウェアを利用して仮想マシンを管理する方式。
- 構築がしやすいため、検証環境などで使われる。
- 仮想マシンでハードウェアを制御する際は、ホストOS を経由する必要がある。
- 代表例：「VMware Workstation Player」、「VMware Fusion」、「Oracle VM VirtualBox」



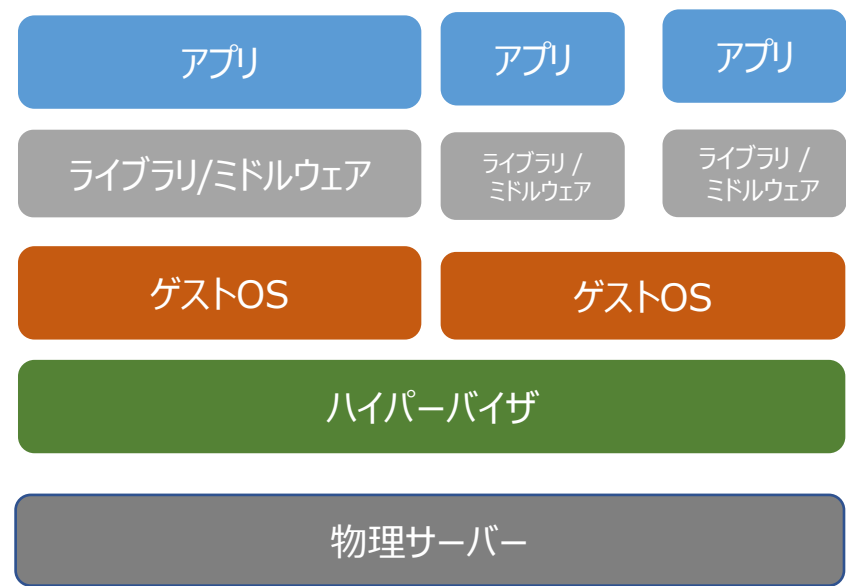
メリット	デメリット
• 既存マシンが利用できる。 • 仮想化に必要なソフトウェアが扱いやすい。 • テスト環境などを作成したいときに便利。 • 既存のOSをホストOSとして利用すればすぐに仮想環境を構築することができる。	• 一般的にハイパーバイザー型の仮想化ソフトに比べて、仮想マシンの動作速度が遅い。 • ホストOSを動作させるための物理リソースが必要。 • ゲストOSを運用する場合にホストOSでの処理速度が限定的になる。 • オーバーヘッド（仮想サーバー上で発生する余分な制御や時間）が大きい。

コンテナ型仮想化

ハードウェアレベル仮想化

ハイパーバイザ（Hypervisor）型仮想化

- ハードウェア上のハイパーバイザ（仮想化のためのOS）を利用して仮想マシンを管理する方式。
- 複数のゲストOSが同時に実行され、各ゲストOSは他のOSから完全に隔離されて動作する。
- あるゲストOSで問題が発生しても、他のゲストOSには影響を与えることはない。
- ホストOSが不要なく、ハードウェアを直接制御するため、仮想マシンの速度低下を最小限に抑える。
- 代表例：「VMware vSphere」、「Microsoft Hyper-V」、「Xen」



メリット	デメリット
- ホストOSが不要でハードウェアを直接制御が可能。 - システム全体の観点から見てリソースの使用効率がよい。 - 管理するサーバの台数削減が可能。	- 仮想化環境の高度な管理を実現するツールが標準装備されていない場合がある。 - ハードウェアのスペックが低かった場合は、処理能力不足となる可能性がある。 - 頻繁にアップデートを要するものもあるため、専門的な知識や技術が必須。

コンテナ型仮想化

OSレベル仮想化

コンテナ型仮想化

- ホストOS上でコンテナと呼ばれる小さな領域でアプリケーション実行環境を提供する方式。
- アプリケーションごとに異なったコンテナを用意し、一つのコンテナで発生した問題が他のコンテナに影響を与えないようにアプリケーションとプロセスを隔離する。
- コンテナはホストOSのカーネルを利用するため、ゲストOSを持たず、処理速度がはやい。
- ベースとなるOSに対応していなければ利用することはできない。
- 代表例：「Docker」、「Kubernetes」



メリット	デメリット
• 必要最小限のCPUやメモリしか使用しないので、負荷が小さく高速な動作が実現可能。 • 比較的容易でコピーも簡単なため、作業時間の大幅な短縮が可能。 • 開発と運用の相性がよくリリースサイクルの高速化が期待できる。	• 複数のホストでのコンテナ運用が煩雑になる。 • カーネルを他のコンテナと共有するため個別に変更できない。 • コンテナサービスのOSはほとんどがLinuxで、異なるOSを同一基板上で動かせない。

コンテナ型仮想化

ホスト型、ハイパーバイザ型、コンテナ型の違い



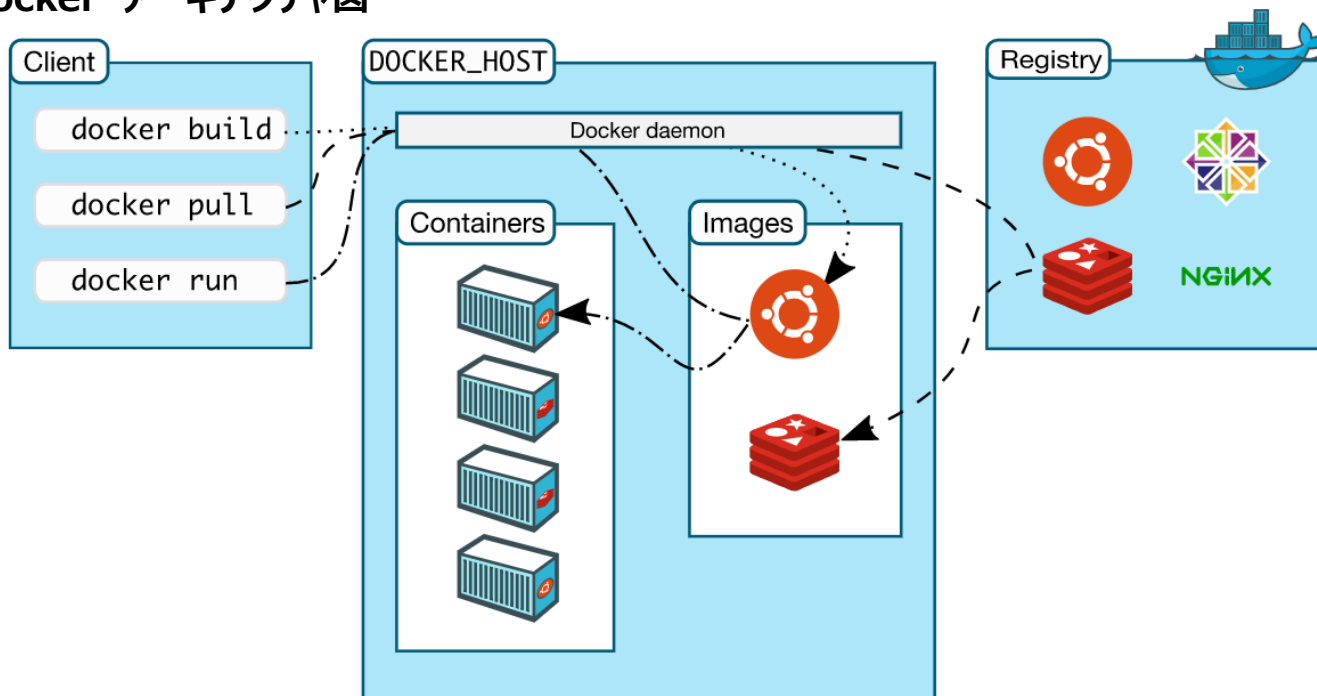
	ホスト型	ハイパーバイザ型	コンテナ型
使用できるOS	何でも可	何でも可	Linuxのみ
リソース	仮想OS毎にCPU、メモリ、ディスクのサイズを割り当てる必要がある	仮想OS毎にCPU、メモリ、ディスクのサイズを割り当てる必要がある	コンテナ毎にCPU、メモリ、ディスクを割り当てず、リソースはコンテナ同士で共有される
オーバーヘッド	仮想OS→仮想化ソフト→ホストOS→物理サーバと、アクセス経路が一番長く、オーバーヘッドが一番大きい	仮想OS→ハイパーバイザー→物理サーバとなり、ホスト型と比べて、オーバーヘッドが少ない	仮想OSが存在しないため、オーバーヘッドがほぼなく、ホスト型、ハイパーバイザ型と比べ、オーバーヘッドが一番少ない。

コンテナ型仮想化

Docker（ドッカー）

- 2013年にDocker社（旧dotCloud）によって開発されたコンテナ型のアプリケーション実行環境を提供するオープンソースソフトウェア。
- ローカル仮想環境とサーバー環境で同じ構成で開発することができる。
- Linuxコンテナ技術を基に構築され、コンテナの作成とビルド、イメージの配布、バージョン管理のプロセスを容易にする。

Docker アーキテクチャ図



コンテナ型仮想化

Docker 構成

- Docker はクライアントサーバシステムで構築されている。

Docker daemon (デーモン)

- Docker API リクエストを受け取り、イメージ、コンテナ、ネットワーク、ボリュームなどのDockerオブジェクトを管理する。
- 他のデーモンと相互に通信を行い、複数のホスト間でコンテナを分散させて管理することもできる。

Docker client (クライアント)

- Dockerを動作させるためのツールやコマンドのこと。
- docker runなどのコマンドを使用すると、クライアントはこれらのコマンドをデーモンに送信し、デーモンがそれらを実行する。

Docker registries (レジストリ)

- DockerレジストリはDockerイメージを保存する場所。
- Docker Hubは誰でも使用できるパブリックレジストリのこと、自身で作成したDockerイメージを保管するだけでなく、他のユーザーが作成したDockerイメージを利用することもできる。

コンテナ型仮想化

Docker Objects

- Docker はイメージ、コンテナ、ネットワーク、ボリューム、プラグイン、およびその他のオブジェクトを作成して使用する。

image（イメージ）

- Dockerコンテナを作成するための手順や構成要素が記載された読み取り専用のテンプレート。
- Dockerレジストリに格納され、ダウンロードして利用したり、他のユーザーと共有することができる。
- 独自のイメージを作成するには、イメージの作成と実行に必要な手順を定義するための簡単な構文を使用して**Dockerfile**を作成する。
- Dockerfile は、どのようなDockerイメージを作成するのかの設定を記述するテキストファイルであり、Dockerfile を配置しておくことで、DockerコマンドでDockerイメージのビルドを行う際、その Dockerfile に従った Dockerイメージを作成することができる。

Container（コンテナ）

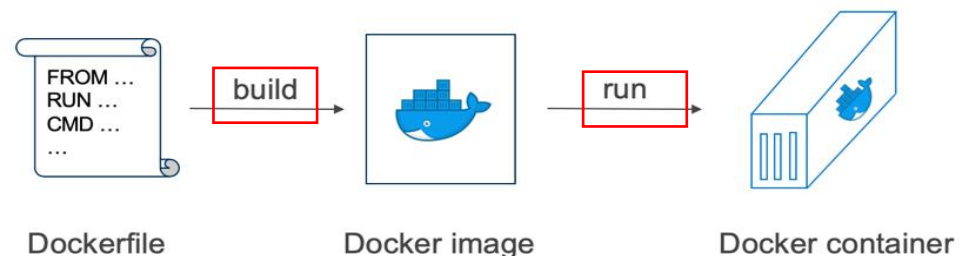
- Dockerイメージから作成された実行環境であり、アプリケーションを実行するための隔離された環境を提供する。
- Docker APIまたはCLIを使用して、コンテナを作成、開始、停止、移動、または削除できる。
- コンテナを1つ以上のネットワークに接続したり、ストレージを接続したり、現在の状態に基づいて新しいイメージを作成したりすることもできる。

コンテナ型仮想化

Dockerfile

Dockerイメージの作成と配布

- Dockerfile を作成することで、Dockerイメージを配布することができる。
- 配布された Dockerfile をビルドすることでDockerイメージが作られ、コンテナを作成することが可能となる。



Dockerfileの中身

コマンド	意味
FROM	Dockerイメージをどこからとってくるのか
RUN	引数に指定したコマンドを実行する
WORKDIR	ディレクトリの指定
COPY	コピーの実行

コンテナ型仮想化

Dockerfile

Dockerfile の作成の流れ

1. ベースイメージの指定
2. イメージ情報を登録
3. パッケージのインストール
4. ソースコードをコピー
5. デフォルト環境変数の指定
6. 実行コマンドの指定

例)

```
FROM python:3.7.5-slim
LABEL author="DeepSquare"
RUN pip install Flask requests
COPY ./server.py /server.py
ENV PORT 80
CMD ["python", "-u", "server.py"]
```

Dockerfileで利用できる構文

コマンド	意味
FROM	利用するイメージの宣言
LABEL	イメージに追加する表示方法
RUN	実行するコマンド
WORKDIR	ディレクトリの指定
COPY	ファイルのコピー
ENV	環境変数のデフォルト値
CMD	コンテナを起動したときに実行されるコマンド

コンテナ型仮想化

Dockerの基本的なコマンド

Docker のバージョン確認

```
$ sudo docker version
```

利用できるDocker イメージの確認

```
$ sudo docker images
```

Docker イメージを探す

```
$ sudo docker search
```

Docker イメージをDocker レジストリから入手する

```
$ sudo docker pull
```

Docker イメージを作成する

```
$ sudo docker build
```

コンテナ型仮想化

Dockerの基本的なコマンド

コンテナの起動

```
$ sudo docker run -it (os) ( もしくはCONTAINER IDまたはNAME)
```

コンテナの停止

```
$ sudo docker stop (CONTAINER IDまたはNAME)
```

コンテナの削除

```
$ sudo docker rm (CONTAINER IDまたはNAME)
```

稼働中のコンテナ名を確認する場合は、「`sudo docker ps`」
稼働中、停止中すべてのコンテナの一覧を確認する場合は、「`sudo docker ps -a`」コマンドを使用する。

コンテナ型仮想化

Docker Compose

- 一度に複数のコンテナを作成、実行できるソフトウェアであり、Dockerイメージのビルドや各コンテナの起動・停止などをより簡単に行えるようにするツール。
- Dockerビルドやコンテナ起動のオプションなどを含めた複数のコンテナの定義を yml ファイルに書き、それを利用してDockerビルドやコンテナ起動をすることができる。

docker-compose.yml の例

```
version: '2' # docker-composeで使用するバージョン
services: #アプリケーションを動かすための各要素
  db:
    image: mysql
  web:
    build: .
    command: bundle exec rails s -p 3000 -b '0.0.0.0'
    volumes:
      - ../myapp
    ports:
      - "3000:3000"
    depends_on:
      - db
```